

Requirement 1 - miFoodi browses food menu (all dishes with price) by various miKitchens/miCooks and places an order as dine-in or take-away

1a. Browsing kitchens - ● Partial

What exists	Gap
Discovery endpoints (nearest, top-rated, recommended, filtered, restaurants/{id}) return kitchen cards with images, ratings, and timing.	The kitchen detail response does NOT include the food menu (dish list with prices). DiscoveryController::show loads addedimage, certificate, weektimings, and user - but not foods relation. A separate GET /api/v2/mymenu exists but is cook-only (behind restaurant middleware). There is no public/foodie-accessible menu endpoint for a given kitchen.
Filter supports dine_in and take_away flags on kitchens.	No per-dish dine_in / take_away flag exists on the foods table or FoodsController. A miFoodi cannot tell whether a specific dish is dine-in only or take-away only.
No keyword / dish-name text search.	Search is ranking-only. A foodie cannot search "biryani" or "pasta" across all kitchens.

1b. Placing an order - ● Critical Missing Feature

What exists	Gap
OrderController::store is fully implemented with validation, pricing, discount, and DB persistence logic.	There is NO route registration for this endpoint in api.php. The method is completely unreachable by any client. The mobile app has no order placement repository or UI. End-to-end order placement is fully broken.
Order model correctly carries dine_in, take_away, persons, delivery_date, delivery_time_from, delivery_time_to.	The mobile app has no ordering flow UI - no cart screen, no order summary screen, no dine-in vs take-away selection screen for the miFoodi.

Requirement 2 - miFoodi saves credit/debit cards & uses them or a one-time card at checkout

2a. Saving cards - 🟡 Partial (backend ready, mobile NOT wired)

What exists	Gap
<code>POST /api/v2/payments/cards</code> - add card. <code>GET /api/v2/payments/cards</code> - list cards. Stripe <code>payment_method_id</code> validated with <code>pm_</code> prefix.	Mobile app <code>payments_repository.dart</code> only calls <code>GET</code> (fetch), never <code>POST</code> (add card). There is no "Add Card" button or UI in the mobile app <code>payments</code> page. The page only shows saved cards and payment history.
<code>POST /api/v2/payments/checkout-session</code> creates a Stripe-hosted card setup session.	Mobile app does not call this endpoint either. No checkout session flow in the app.

2b. Selecting a saved card at checkout - 🔴 Not implemented

What exists	Gap
<code>PaymentService::resolveCustomerPaymentMethodId</code> supports resolving by saved <code>card_id</code> or raw <code>pm_</code> string. <code>createPaymentIntent</code> supports creating an intent against a customer.	The <code>createIntent</code> endpoint (<code>POST /api/v2/payments/intent</code>) does NOT accept a <code>card_id</code> or <code>payment_method_id</code> parameter. It only takes <code>order_id</code> . The card is only specified at <code>confirmIntent</code> . The flow does not support "select card → pay" in a single action.
	Mobile app has NO checkout/order-payment flow UI. miFoodi cannot select a card during ordering.

2c. One-time payment with a different card - ● Not implemented

What exists	Gap
Stripe PaymentIntent system is flexible enough to support a one-time <code>payment_method_id</code> on confirm.	No API endpoint or mobile UI allows passing a one-time card (new <code>pm_*</code>) at order time. The <code>confirmIntent</code> endpoint uses only the <code>card_id</code> stored on the <code>Payment</code> record, set during intent creation - which does not currently accept a card reference.

Requirement 3 - miFoodi can cancel the order until it is not yet accepted by miCook

3a. Cancellation API - ● Incorrect scoping

What exists	Gap
<code>POST /api/v2/updateorderstatus</code> allows setting status to 4 (cancelled). <code>canManageOrder</code> allows both the order owner (user) and the owning cook to change status.	There is no guard preventing the miFoodi from cancelling an already-accepted (status=3/confirmed) order. The <code>statusUpdate</code> method only validates the allowed values; it does not enforce the business rule "cancel only if not yet accepted". A foodie can cancel a confirmed/in-progress order.
	There is no cancellation reason or policy enforcement (e.g., no-show window, cancellation fee). The <code>CancelReason</code> model exists but nothing in <code>statusUpdate</code> requires or records a cancel reason.
	No push notification is sent to the miCook when a miFoodi cancels. <code>FcmController::sendTo</code> exists but is never called from <code>OrderController</code> .

Requirement 4 - miCook sees incoming orders, accepts them; payment deducted; status → confirmed / in-progress

4a. Accepting and triggering payment - Partial / incorrect

What exists	Gap
POST /api/v2/updateorderstatus with status=3 triggers PaymentService::confirmPaymentIntent . Payment record is locked and confirmed in a DB transaction. order.paid=1 and order.status=3 are set.	Payment can only be confirmed if a Payment record already exists for the order. But with order placement broken (Req 1), no Payment record will exist. The flow is: place order → create intent → cook accepts → payment confirmed. The middle step (create intent) is also not called from mobile.
	After acceptance, no push notification is sent to the miFoodi confirming acceptance, pickup/dine-in time, or estimated wait. FcmController is orphaned - never invoked in the order lifecycle.
	The status after acceptance is 3 (confirmed). The codebase has no "in progress" status (e.g., status=5). The FAQ and website reference "in progress" semantically, but no status exists for it.

Requirement 5 - miFoodi receives order confirmation and dine-in/pick-up time

5a. Confirmation notification - Not implemented

What exists	Gap
account/device-token endpoint exists to store FCM tokens. FcmController::sendTo exists with legacy FCM v1 HTTP API.	FcmController::sendTo is never called from OrderController or statusUpdate . No push notification is sent at any point in the order lifecycle to either party.
Order model carries delivery_date , delivery_time_from , delivery_time_to .	These fields are set at order creation time, not when the cook accepts. There is no mechanism for the cook to propose or override the pickup/dine-in time at acceptance.

5b. Order detail visible to miFoodi - Partial

GET /api/v2/account/orders lists miFoodi's orders with status. But with order placement broken, there will be no orders to see.

Requirement 6 - miCook can see orders and view pick-up/dine-in time

6a. - Implemented (with caveats - fix the caveat)

kitchenupcomingorders , kitchenorderrequest , allorders all return
delivery_date , delivery_time_from , delivery_time_to via the Order resource.
Filter by take_away / dine_in sort exists.

Caveat: There is a **typo in the method name** - myUpcomingOrderss (double s). The route maps to it correctly but it is a maintenance risk.

-> Fix the method name to myUpcomingOrders and also all references to this method should be updated. there MUST not be any method name or reference or method call or instantiation etc. for myUpcomingOrderss.

Requirement 7 - miCook creates daily/weekly menu; sets dine-in seats; creates dine-in timeslots

7a. Daily/weekly menu - Not implemented as designed

What exists	Gap
<code>FoodsController</code> allows adding/editing/toggling individual food items with <code>food_name</code> , <code>price</code> , <code>cookingstyle</code> , <code>specialDiet</code> .	There is no concept of daily or weekly menu scheduling. A dish is just either active or inactive globally. There is no <code>available_date</code> , <code>available_days</code> , <code>available_from_time</code> , <code>available_to_time</code> field on <code>foods</code> . A miCook cannot say "I serve lamb on Fridays only" or "this dish is only available on weekends".
FAQ says "availability schedule" and "pre-order meals" are features.	No scheduling schema, validation rule, or API field supports this. The website FAQ states this as a user-facing promise that is not implemented.

7b. Dine-in seats - Partial

What exists	Gap
<code>Mikitchn.no_of_seats</code> field exists and is set during <code>createKitchen</code> / <code>updateKitchen</code> .	No enforcement at order creation time. <code>OrderService::createOrder</code> does not check if <code>persons</code> exceeds <code>kitchen.no_of_seats</code> . A miFoodi could theoretically book more people than the kitchen capacity.
	No real-time availability tracking - no check for concurrent bookings that might exceed capacity for the same date/time slot.

7c. Dine-in timeslots - Not implemented

What exists	Gap
Kitchen Timing model and weektimings relation store open/close hours per weekday.	There is no time-slot system. No <code>DineInSlot</code> model, no slot capacity, no slot booking. The order just carries a free-form <code>delivery_time_from</code> / <code>delivery_time_to</code> window.
FAQ Q10 says "Choose the available date and time" implying a slot picker.	No slot availability API exists. The mobile app has no slot picker UI. miFoodi cannot choose from available slots - they enter a time window manually (same pattern as the lat/lng location input problem).

Changes Implemented

A. Fix the Critical Blocker First (P0)

Register `OrderController::store` in `api.php` under a customer-gated route. Without this, the entire platform value proposition (placing orders) does not function. This is a 1-line fix in `api.php` . `Route::post('orders', [OrderController::class, 'store'])->middleware('customer');`

B. Introduce a Proper Order Lifecycle with Notifications

The FCM infrastructure exists but is dormant. Wire push notifications into every state transition:

Event	Who to notify	Message
Order placed	miCook	"New order request from [Foodie Name]!"
Cook accepts	miFoodi	"Your order is confirmed! Come at [time]."
Cook declines	miFoodi	"Your order was declined. Full refund initiated."
miFoodi cancels	miCook	"Order cancelled by customer."

Event	Who to notify	Message
Order ready	miFoodi	"Your meal is ready for pickup/dine-in."
Order completed	Both	"Thanks for using mitab! Leave a review."

C. Slot-Based Dine-In Booking System

Instead of free-form time windows, introduce a proper slot model:

- miCook sets **available time slots** (e.g., 12:00–13:00, 13:00–14:00) with **seat capacity per slot**
- miFoodi sees only open slots and picks one from a picker
- Backend validates: `persons ≤ remaining_capacity_for_slot`
- Concurrent bookings handled with a pessimistic lock or slot reservation token

D. Food Menu Scheduling (Daily/Weekly)

Add an `availability` concept to the `foods` table:

- `available_days` (array of weekdays, e.g., `["Friday", "Saturday", "Sunday"]`)
- `available_from` / `available_to` time
- This allows a miCook to set "I serve Sunday roast only on Sundays"
- Discovery API surfaces only dishes available on today's date/day

E. Dish-Level Dine-In / Take-Away Tagging

The current system only tags the `kitchen` as dine-in/take-away. Add per-dish flags:

- `dine_in_available` : boolean on `foods`
- `take_away_available` : boolean on `foods`
- Discovery `restaurants/{id}` response should include the menu with these flags so miFoodi can filter dishes by their preferred service type

F. Pre-Order / Advance Booking

The FAQ mentions "pre-order meals." Implement:

- Ability to order X days in advance (configurable per cook)
- Cook sets `advance_order_days` in kitchen profile
- Validation in `OrderController::store`: `delivery_date ≤ today + advance_order_days`

G. No-Show & Cancellation Policy

The FAQ mentions "reservation fees" for no-shows. Implement:

- A **hold/authorise** Stripe PaymentIntent at order placement (not immediate capture)
- **Cancel policy window**: free cancellation within N hours of placing; penalty afterward
- Cook's no-show flag marks a booking as no-show → triggers partial payout to cook
- Currently, payment is only captured on cook acceptance (`capture_method: automatic`). Switch to `capture_method: manual` → capture on acceptance, refund on cancel.

H. Keyword Search for Dishes Across Kitchens

Currently there is no text search. Add:

- `GET /api/v2/discovery/search?q=biryani` - searches across `foods.food_name` + `foods.description` and `mikitchens.name`
- Returns matched dishes grouped by kitchen with availability and price

I. In-App Messaging Between miFoodi and miCook

FAQ Q15 mentions "you may be able to message the cook through the mitabl app." Currently, no messaging system exists:

- A basic thread-per-order messaging model (similar to the existing support ticket pattern) would fulfill this
- Useful for dine-in instructions, dietary preferences, late arrivals

J. Account Delete Flow

Mobile app calls `v2/account/delete` but the backend route does not exist. This is a required feature for both App Store and Google Play compliance (mandatory user data deletion). Wire up a proper GDPR/app store-compliant account deletion endpoint.

K. Security: Registration Endpoint is Open

As noted in `docs/Planned Improvements.md`, anyone can register unlimited accounts (cook/foodie) with no rate limiting or device verification. For a food-trust platform this is significant:

- Add throttle middleware to `POST /api/register`
- Consider email domain or phone number uniqueness enforcement
- Optionally, add an app-signature header check to prevent bots

L. Hardcoded Business Logic Should Be Config-Driven

From `OrderService`:

- Discount for first 5 orders: `$discountAmountCents = 5000` (\$50 AUD) - hardcoded
- GST rate: `10` - hardcoded in two places

These should be pulled from `PlatformSettingRegistry` (already exists) so the admin can adjust without a deployment.

M. miCook Identity & Trust Signals (Platform Philosophy)

The platform is built on community trust. Consider surfacing:

- **Cook story/bio** - `user.description` exists but is not consistently surfaced in discovery cards
- **Years cooking / Cuisine specialties** - not captured in the data model
- **Verification badge** - certificate/ABN status is stored but not visually surfaced in the discovery feed
- **Repeat customer indicator** - tell a miFoodi "You ordered from here before" (helps retention)

N. Review Flow is Route-Orphaned

`ReviewController` (add review to restaurant, add review to foodie, get kitchen reviews) exists fully - but **none of its routes are registered in `api.php`** . Reviews are referenced in discovery (ratings from `reviews.avg_rating`) but foodies cannot submit reviews. This breaks the "ratings and reviews" trust mechanic mentioned in FAQ Q5 and Q6.